指针

指针变量：指向内存地址

指针变量定义格式

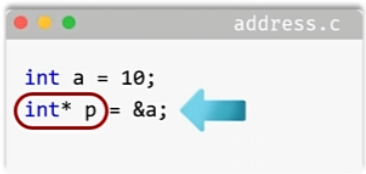
数据类型 *变量名

指针的作用：

利用指针查询，修改变量数据

定义一个变量，会有地址出现，定义一个指针，里面存的是这个另外一个变量的地址

*是解引用运算符，获取这个地址的值



```
address.c

int a = 10;
int* p = &a;

printf("%d\n",*p); //10
```

定义指针的时候，*只是一个标记符号，表示当前是一个指针，里面记录内存地址

使用*的时候是解引用运算符，表示查询这个内存地址的数据

*p = 200；表示存储一个数据

注意事项

指针的名字不带* p才是他的名字，指针的数据类型要和指向的数据类型一致，指针本身也会占用内存空间，跟数据类型无关，跟编译器有关，表示的是地址，32位：4字节，64位：8字节。

指针赋值的时候，不能把数值赋给指针变量，给的是地址。

操作其他函数里面的变量

如果不用指针，操作函数里面变量的话，num1 num2 这两个函数的参数，只是传递了a和b的值进去也就是把10 和20 传进去，并没有交换ab的值，交换的是num这个临时变量的值

代码分析

pointer.c

```c
int main()
{
    int a = 10;
    int b = 20;
    swap(a,b);
    return 0;
}
```

pointer.c

```c
void swap(int num1, int num2)
{
    int temp = num1;
    num1 = num2;
    num2 = temp;
}
```

```c
void swap(int* p1, int* p2);
int main()
{

    int a = 10;
    int b = 20;


    int* p1 = &a;
    int* p2 = &b;
    swap( p1, p2);


    printf("%d,%d\n", *p1 ,*p2);
```

把地址传进去，也可以用取地址符号&

```c
void swap(int *p1, int *p2)
{
    int temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}
```

细节

局部变量会在函数结束之后内存回收，变量直接没有值，定义一个返回int*的指针，如果不加static关键字，返回的值是随机的

定义int*类型的指针变量返回值，返回的是要地址而不是直接的内存，因此使用这个函数的时候，需要带*号取出这个指针地址的值

```c
int*  swap(void)
{
    static int a = 10;
    return &a;
}
```

```c
    printf("%d\n",  *swap());
```

利用指针返回函数多个值

定义一个函数，指针类型的变量，解引用出来赋值，然后在判断这个值是不是最大最小值

传递进去的时候，要把变量地址传递进去，所以用取地址符号

```
void  swap(int arr[] , int len ,int* max ,int *min)
{
    *max = arr[0];
    *min = arr[0];
    for (int i = 1; i < len;i++)
    {
        if (arr[i] > *max)
        {
            *max = arr[i];
        }
    }

    for (int i = 1; i < len;i++)
    {
        if (arr[i] < *min)
        {
            *min = arr[i];
        }
    }

    return;
```

```
void  swap(int arr[], int len, int* max, int* min);
int main()
{



    int arr[] = { 0,1,2,2222,5,7,9,10,15,10 };
    int len = sizeof(arr)/sizeof(int);
    int a = 0;
    int b = 0;
    swap(arr, len, &a, &b);

    printf("%d\n", a);
    printf("%d\n", b);
```

利用指针将函数的结果和状态分开

可以设计函数，返回值是当前函数执行状态，用指针取出函数计算结果，设计指针参数，函数内部的*是取地址的值，把计算结果赋到这个地址的值。那么函数的返回值可以判断当前这个函数的状态，达到目的，所以函数使用的时候，也需要把地址传递进去

```c
int  swap(int num1 , int num2 ,int* res)
{
    if (num2 == 0)
    {
        return 1;
    }
    *res = num1 % num2;


    return 0;

}
```

```c
    int a = 10;
    int b = 3;
    int res = 0;

    if (swap(a, b, &res) == 0)
    {

        printf("%d\n", res);
    }
```

指针的计算

指针的类型是什么意思？

数据类型 * 变量名 = 内存地址

int * p = 0x001

通过定义指针的内存地址，只能取到地址首字节，无法获取后面的值，因此这个和数据类型相关，指针数据类型和指针获取字节个数息息相关。

步长是指针移动了一次，走了一个字节。一个地址有1个字节，所以4个字节就是移动4个地址。

1. 指针 + 1 或者指针 - 1 是什么意思？

   把指针中记录的内存地址，往后或者往前移动一个步长

2. 什么是步长？跟什么有关？

   跟数据类型有关
   Windows64位操作系统：
   char：移动一个字节
   short：移动两个字节
   int：移动四个字节
   long：移动四个字节
   long long：移动八个字节

有意义的操作

指针跟整数进行加减操作（每次移动N个步长）

指针跟指针进行减操作（间隔步长）指针与指针之间的减法操作，会求出之间间隔多少个步长

无意义的操作

指针跟整数进行乘除操作（指针指向不明）

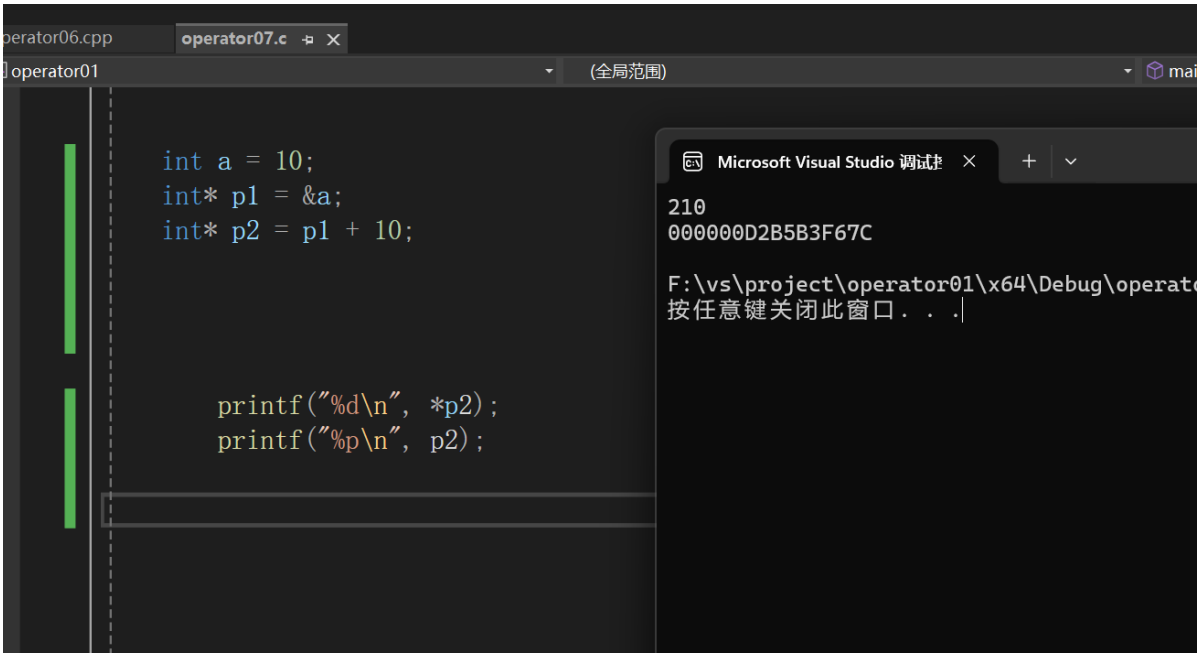
指针和指针进行加乘除操作（意义不明）

指向不明的指针（野指针、悬空指针）

野指针：指针指向空间未分配

悬空指针：指针指向的空间已分配，但是被释放

野指针例子



悬空指针

当函数调用的时候，定义一个数据，然后用指针指向这个地址，但是函数结束后，这个地址的值被回收，当指针再去的时候值就改变了

```
int a = 10;
int* p1 = &a;
int* p2 =swap();

    printf("%d\n", 21132213);
    printf("%d\n", *p2);
    printf("%p\n", p2);

return 0;
}

int*  swap(void)
{
    int num = 10;
    int* p = &num;
    return p;
}
```

```
Microsoft Visual Studio 调试控
21132213
9
000000E8D68FF404

F:\vs\project\operator01\x64\Debug\operator01.exe (进程 29520)已退出，代码为
按任意键关闭此窗口. . .
```

需要用上关键字 static

```
int a = 10;
int* p1 = &a;
int* p2 =swap();

    printf("%d\n", 21132213);
    printf("%d\n", *p2);
    printf("%p\n", p2);

return 0;
}

int*  swap(void)
{
    static int num = 10;
    int* p = &num;
    return p;
}
```

特殊类型指针，没类型的指针

特殊类型指针，void *p 不表示然后类型，无法进行加减操作，无法获取变量里面数据，但是可以接受任意地址

不同类型的指针之间无法互相赋值，有void类型指针可以接受任意内存接受地址，但是void类型的指针无法获取里面的数据，也无法进行加减计算

常用于交换数据函数

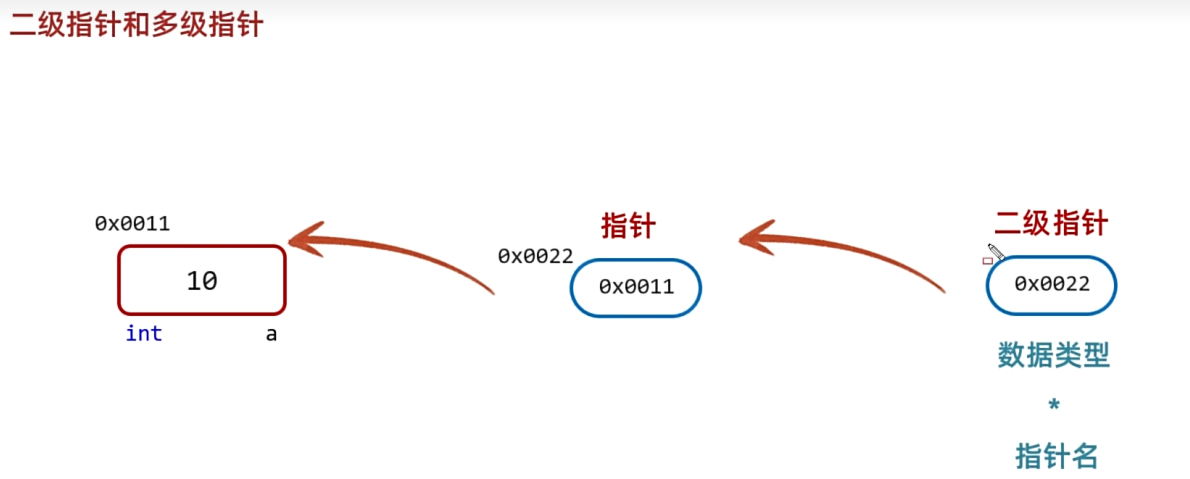
利用void* 指针进行数据交换

```
int c = 100;
int d = 200;
swap(&c, &d, 4);
    printf("%d\n", c);
    printf("%d\n", d);


return 0;
}


void swap(void* p1 , void* p2,int lent)
{

    char* pc1 = p1;
    char* pc2 = p2;

    char temp = 0;
    for (int i = 0; i < lent; i++)
    {
        temp = *pc1;
        *pc1 = *pc2;
        *pc2 = temp;

        pc1++;
        pc2++;

    }
```

```
200
100

F:\vs\project\operator01\x64\Debug\oper
按任意键关闭此窗口. . .
```

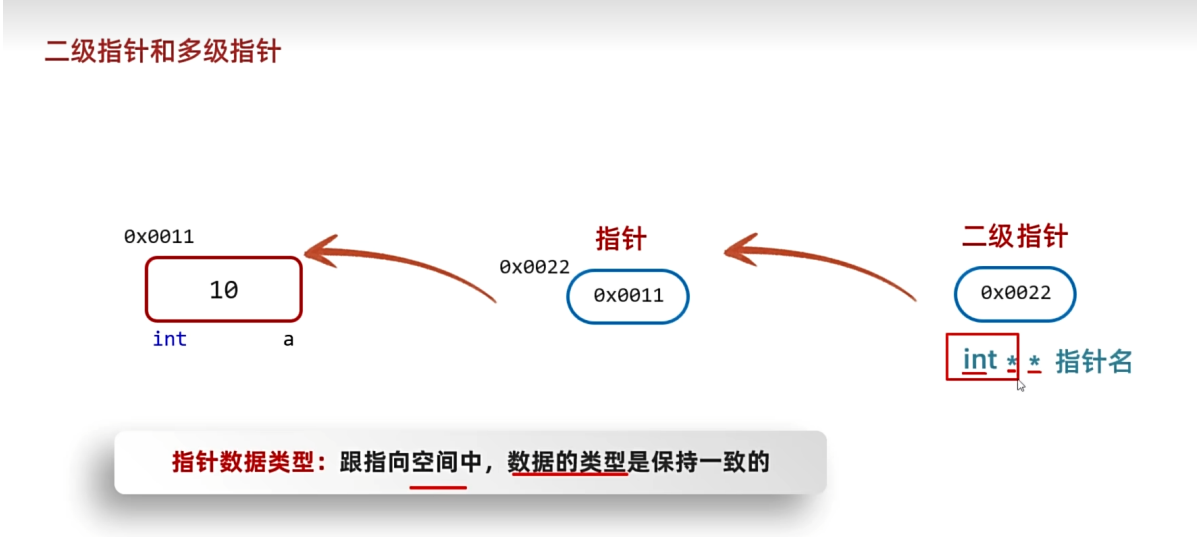
定义两个void* 指针p1 p2 ，在函数内定义char类型的指针pc1 pc2 接收p1 p2 这两个空的类型指针，这两个指针就变成char* 类型指针占1个字节，在for循环内，利用指针里面的值，从第一个字节增到len长度的内存地址，因此就可以满足任意类型数据类型转换。

二级指针和多级指针



指针的数据类型：跟指向空间中，数据的类型保持一致，第二个*是定义指针必须用到的标识，第一个 * 代表着int *类型

二级指针可以操作一级指针内部记录的内存地址



二级指针可以修改一级指针内部记录的内存地址

```
void swap(void* p1, void* p2, int lent)

int main()
{
    int a = 10;
    int b = 11;
    int* p1 = &a;
    int** p2 = &p1;

    *p2 = &b;
    printf("%p\n", &a);
    printf("%p\n", &b);
    printf("%p\n", p1);


    return 0;
}
```

```
00000025B8CFF6A4
00000025B8CFF6C4
00000025B8CFF6C4

F:\vs\project\operator01\x64\De
按任意键关闭此窗口. . .
```

因此要解引用需要用**来进行取值

```
void swap(void* p1, void* p2, i

int main()
{
    i
    i
    int* p1 = &a;
    int** p2 = &p1;

    *p2 = &b;
    printf("%p\n", &a);
    printf("%p\n", &b);
    printf("%p\n", p1);
    printf("%d\n",  **p2);

    return 0;
}
```

int main()
联机搜索

```
0000003C2AB7F624
0000003C2AB7F624
11

F:\vs\project\operator01\x
按任意键关闭此窗口. . .
```

数组和指针

指向数组的指针，叫做数组指针

用循环指针遍历数组

```
int arr[] = { 10, 20,30,40 ,50 };
int len = sizeof(arr) / sizeof(int);
//两个地址
int* p1 = arr;
int* p2 = &arr[0];


for (int i = 0; i < len;i++)
{
    printf("%d\n", *(p1 + i));

}
return 0;
}
```

注意事项

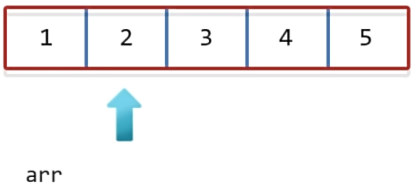
数组指针在计算的时候，平时代表着整个整体，但是参与计算的时候，会退化成为第一个元素的指针

数组指针的细节

- arr参与计算的时候，会退化为第一个元素的指针

pointer.c

```
int arr[] = {1,2,3}
int* p = arr;
int* pp = arr + 1;
```



特殊情况 sizeof 不会退化，arr还是整体

&arr 获取地址的时候，表示的还是整体

因此在参与运算的时候 arr 与 &arr都表示数组第一个值的地址

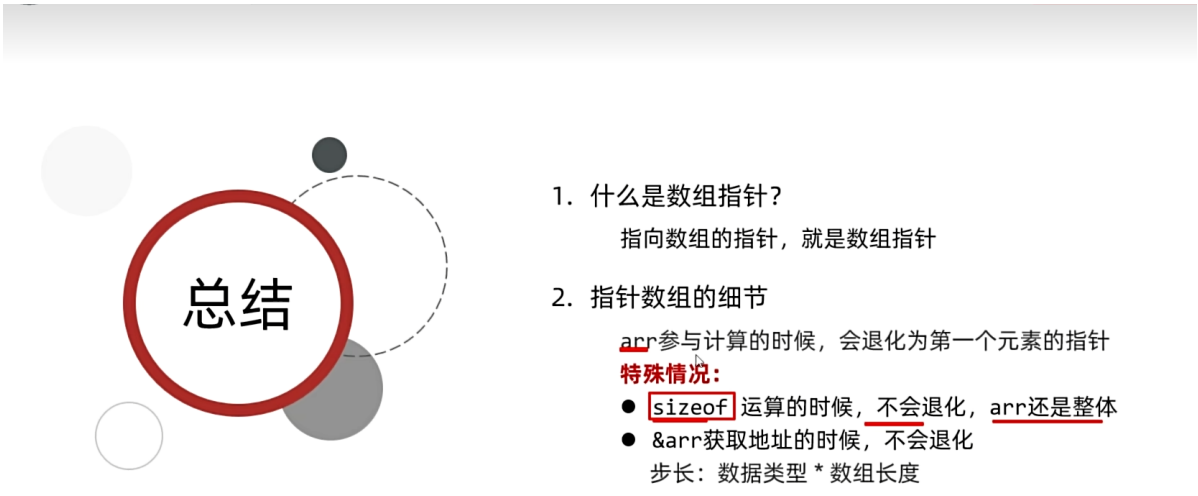
但是步长不一样

```
// 1.定义数组
int arr[] = {1,2,3,4,5,6,7,8,9,10};

// 2.sizeof 运算的时候，不会退化，arr还是整体
printf("%zu\n", sizeof(arr));// 40

// 3.&arr获取地址的时候，不会退化，记录的内存地址第一个元素的首地址，也是数组的首地址，步长：数据类型 * 数组的长度
// arr参与计算的时候，会退化为第一个元素的指针,记录的内存地址第一个元素的首地址，也是数组的首地址，步长：数据类型 int 4
```

```
int arr[] = { 1, 2,3,4 ,5,6,7,8,9,10};
int len = sizeof(arr) / sizeof(int);
//两个地址
int* p1 = arr;
int* p2 = &arr;

printf("%p\n", arr);
printf("%p\n", &arr);

printf("%p\n", arr+1);
printf("%p\n", &arr+1);
```

Microsoft Visual Studio 调试

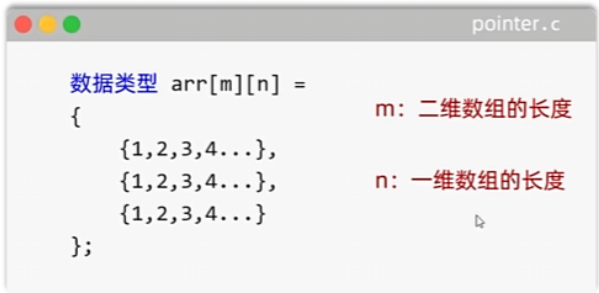
```
000000F1C4AFFBE8
000000F1C4AFFBE8
000000F1C4AFFBEC
000000F1C4AFFC10

F:\vs\project\operator01\x64\
按任意键关闭此窗口. . .
```



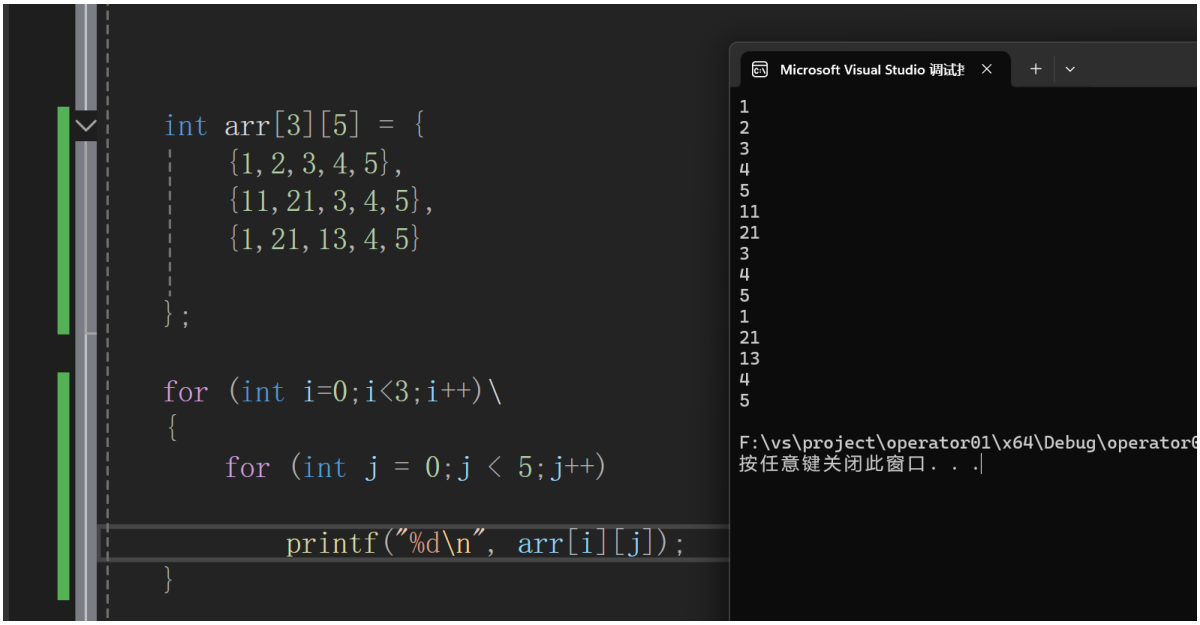
二维数组

概念：把多个小数组，放到一个大数组之中

命名规则



利用for循环遍历二位素组 缺点：二维数组中一维素组长度都必须一样



第二种命名方式用指针的方式，三个数组长度不一样，命名二维数组的话，因为在命名二维数组的时候，内部参数是arr1，也就是取到arr1的指向第一个元素指针，那么二维数组的数据类型，也要和arr1这个数据量的参数一致，也需要用指针，取得是arr1第一个数据的地址步长是4

```
int arr1[3] = { 1,2,3 };
int arr2[5] = { 1,2,3,4,5 };
int arr3[9] = { 1,2,3 ,4,5,6,7,8,9};

int len1 = sizeof(arr1) / sizeof(int);
int len2 = sizeof(arr2) / sizeof(int);
int len3 = sizeof(arr3) / sizeof(int);
int lenarr[3]={ len1 , len2 ,len3 };

int* arr[3] = { arr1,arr2,arr3 };
```

遍历不同长度的二维数组

```
int arr1[3] = { 1,2,3 };
int arr2[5] = { 1,2,3,4,5 };
int arr3[9] = { 1,2,3 ,4,5,6,7,8,9};

int len1 = sizeof(arr1) / sizeof(int);
int len2 = sizeof(arr2) / sizeof(int);
int len3 = sizeof(arr3) / sizeof(int);
int lenarr[3]={ len1 , len2 ,len3 };

int* arr[3] = { arr1,arr2,arr3 };

for (int i = 0;i < 3;i++)
{
    for (int j = 0;j < lenarr[i];j++)
    {

        printf("%d\n", arr[i][j]);
    }
    printf("\n");
}
```

```
Microsoft Visual Studio 调试控
1
2
3

1
2
3
4
5

1
2
3
4
5
6
7
8
9


F:\vs\project\operator01\x64\Debug\operator01.e
按任意键关闭此窗口. . .
```

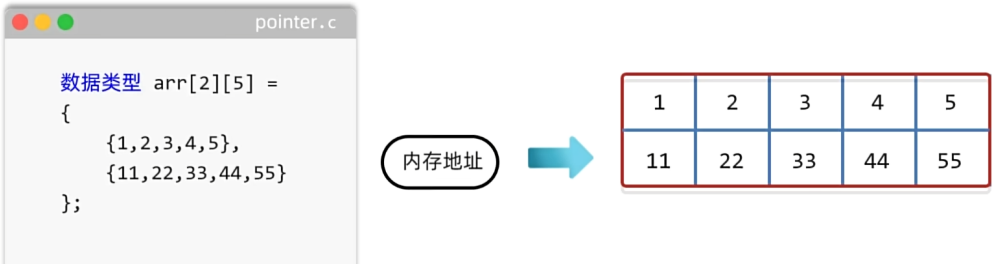
用指针遍历二维数组

二维数组的指针

命名规则

数据类型* 指针名 =arr

int[5]* p 命名规范 int(*p)[5]




```
int main()
{

    int arr[3][5] =
    {
        {1, 2, 3, 4, 5},
        {1, 22, 322, 422, 5},
        {1, 2222, 322, 422, 51},
    };
    int (*p) [5]= arr;
```

设定一个二维数组的指针，如何去遍历里面的值

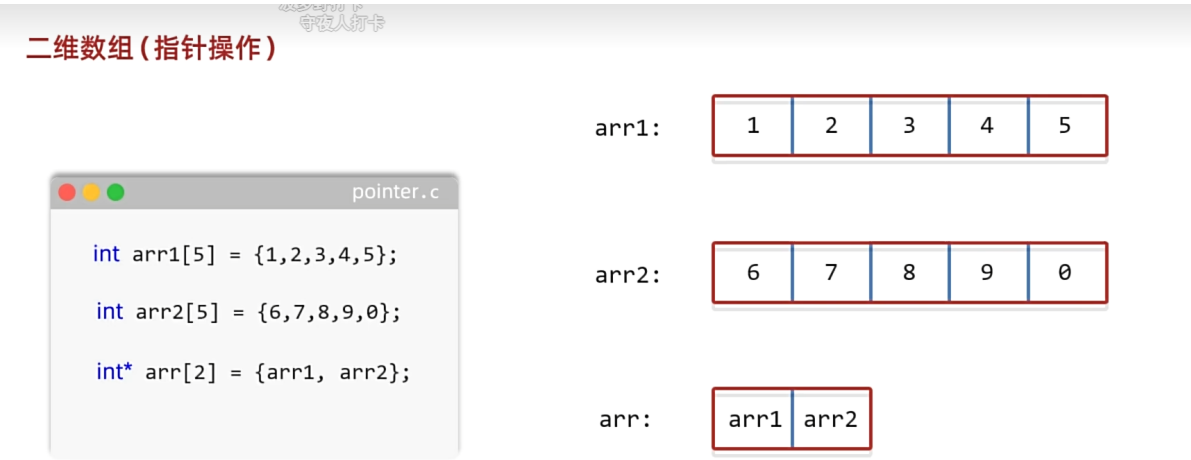
在一开始，解引用运算符得到的是一位数组的指针，步长是一个字节，然后再解引用运算符，这样才算出来是这个位置的值，并且在每次内循环结束后，二维数组的指针要 + 1，步长是一位数组的长度，这样才能跳到第二个一位数组的位置。

```
int arr[3][5] =
{
    {1, 2, 3, 4, 5},
    {1, 22, 322, 422, 5},
    {1, 2222, 322, 422, 51},
};
int (*p) [5]= arr;

for (int i = 0;i < 3;i++)
{
    for (int j = 0;j < 5;j++)
    {

        printf("%d\n", *(*p + j));
    }
    printf("\n");
    p++;
}
```

```
1
2
3
4
5

1
22
322
422
5

1
2222
322
422
51

F:\vs\project\operator01\x64\Debug\operator01.exe (
按任意键关闭此窗口. . .
```

用指针遍历 二维数组（指针命名）

两个一位数组的内存地址是分开的，并不是连续一段空间，只是把一维数组的指针放到了二维数组中，区别于上面直接定义，二维数组只存放内存地址



命名规范 *p是定义指针的标记，int *是数据类型，表示是一个内存地址，因为二维数组命名的时候，int * arr[2]=。。。代表命名数据类型就是int *类型也就是指针代表arr1的内存首地址

int* *p =arr

```
int arr1[5] = { 1, 2, 3, 4, 5 };
int arr2[5] = { 21, 2, 322, 422, 5 };
int arr3[5] = { 1, 222, 223, 4, 5 };

int* arr[3] = { arr1, arr2, arr3 };

int** p1 = arr;

for (int i = 0;i < 3;i++)
{
    for (int j = 0;j < 5;j++)
    {
        printf("%d\n", *(*p1 + j));
    }
    p1++;
}
```

```
Microsoft Visual Studio 调试控
1
2
3
4
5
21
2
322
422
5
1
222
223
4
5

F:\vs\project\operator01\x64\Debug\oper
按任意键关闭此窗口. . .
```

数组指针，指针数组

数组指针：指向数组的指针，方便操作数组里面各种各样的数据

int* p = arr;　　步长为：int 4个字节

int (*p)[5]=&arr;　　*步长为：int* * 5 20个字节

指针数组：存放指针的数组

用来存放指针

```
pointer.c
int arr1[5] = {1,2,3,4,5};
int arr2[5] = {6,7,8,9,0};
int* arr[2] = {arr1, arr2};
```

函数指针

格式：返回值类型（*指针名）（形参列表）

作用：利用函数指针，动态调节函数

```
void method1();
int method2(int num1, int num2);
int main()
{
    void (*p1)()=method1;
    int (*p2) (int , int ) = method2;
```

调用p1

```
void method1();
int method2(int num1, int num2);
int main()
{
    void (*p1)()=method1;
    int (*p2) (int , int ) = method2;
    p1();
}

void method1()
{
```

```
Microsoft Visual Studio 调试
method1

F:\vs\project\operator01\x64\Debug\operator01.e
按任意键关闭此窗口. . .
```

调用p1 p2

```c
int method2(int num1, int num2);
int main()
{
    void (*p1)()=method1;
    int (*p2) (int , int ) = method2;
    p1();
    int num = p2(2, 2);
    printf("%d",num);
}


void method1()
{
    printf("method1\n");
```

```
Microsoft Visual Studio 调试控
method1
method2
4
F:\vs\project\operator01\x64\Debug\operator01.exe (进程
按任意键关闭此窗口. . .
```

定义数组去装四个函数，函数指针数组（形参和返回值都要完全一样）

```c
#include <time.h>

int mutiply(int num1, int num2);
int subtract(int num1, int num2);
int divide(int num1, int num2);
int add(int num1, int num2);
int main()
{
    int (*p1[4])(int, int) = { add ,subtract ,mutiply ,divide };
```

```c
int divide(int num1, int num2);
int add(int num1, int num2);
int main()
{
    int (*p1[4])(int, int) = { add ,subtract ,mutiply ,divide };
    int num1 = 0;
    int num2 = 0;
    int choose = 0;
    scanf("%d%d", &num1, &num2);


    printf("%d\n", num1);
    printf("%d\n", num2);
    scanf("%d", &choose);
    int sum = (p1[choose - 1])(num1, num2);

    printf("%d\n", sum);
```